Submission Guidelines

Submit the following as part of your assignment:

1. Source Code: **A Jupyter notebook in PDF format along with output for each cell is expected.**
2. Input Data: Sample input(s) used to demonstrate the system or dataset given in the problem statement.

   If the link is expired by the time you access it, search for the similar dataset and use it for the tasks. (only column name changes)
3. Output: Generated output(s), such as screenshots, audio, video, or other relevant results, depending on the assignment.
4. Project Report (1page): The report should include:
   - Objective of the project
   - System architecture or workflow
   - Tools, libraries, or technologies used
   - Implementation details and key features
   - Results and observations
   - Challenges faced (if any)
   - Sample output screenshots

Note: The dataset provided (if any) is for reference only and is not mandatory. You are free to use any appropriate dataset

Evaluation Rubric (13 Marks)

Criteria	Marks
Code quality and documentation	6
Project report	2
Result screenshot of Bits virtual Lab	**2+2(2 mark will be provided for using BITS Virtual lab)**
Innovation/Additional features	**1**
Total	**13**

1: Speech Emotion Recognition

Objective

Develop a system that identifies the emotion expressed in speech.

Dataset

- RAVDESS
- TESS
- CREMA-D

Tasks

1. Collect the dataset.
2. Perform preprocessing.
    - Remove silence
    - Normalize audio
    - Resample to 16 kHz
3. Extract features.
    - MFCC
    - Chroma
    - Mel Spectrogram
    - Zero Crossing Rate
    - Spectral Centroid
4. Train at least three models.
    - SVM
    - Random Forest
    - CNN/LSTM
5. Compare performance.
6. Build a Streamlit interface where users upload speech and receive the predicted emotion.

2.Speaker Identification System

Objective

Identify which registered speaker is speaking.

Dataset

- VoxCeleb
- LibriSpeech
- Self-recorded voices

Tasks

1. Record speech from multiple speakers.
2. Extract MFCC features.
3. Train
    - KNN
    - SVM
    - CNN
4. Predict speaker identity.
5. Compare different classifiers.

Evaluation

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score

---

3. **Keyword Spotting**

Objective

Detect predefined keywords such as

- Yes

- No
- Stop
- Go
- Left
- Right

Dataset

Google Speech Commands Dataset

Tasks

1. Data preprocessing
2. Feature extraction
3. Train CNN
4. Real-time prediction using microphone
5. Display detected command

---

4. **Personalized Text-to-Speech System (Recommended)**

Objective

Develop a system that converts user-provided text into natural-sounding speech using different voices.

Features

- Enter text
- Choose male/female voice
- Adjust speaking rate
- Adjust pitch (if supported)
- Save synthesized speech as a WAV or MP3 file

Technologies

- Python
- Coqui TTS or Piper TTS
- Streamlit

- Librosa (optional)

Evaluation

- Mean Opinion Score (MOS) from users
- Response time
- Audio quality

5.Story Narration System

Objective

Develop a Personalized Text-to-Speech (TTS) system that converts user-provided text into natural-sounding speech with customizable voice and speech settings.

Requirements

Your system should perform the following tasks:

1. Accept text input from the user.
2. Convert the input text into speech using a Text-to-Speech (TTS) engine.
3. Allow the user to select different voices (e.g., male/female or available system voices).
4. Allow the user to adjust the speaking rate.
5. Allow the user to adjust the pitch (if supported by the selected TTS engine).
6. Play the synthesized speech within the application.
7. Save the generated speech as an audio file (e.g., .wav or .mp3).

Expected Features

- Custom Text Input
    - Accept text entered by the user for speech synthesis.
- Voice Selection
    - Allow users to choose from different available voices (e.g., male/female or multiple TTS voices).
- Speech Customization

    - Adjust speaking rate.
    - Adjust pitch (if supported).
- Audio Output
    - Generate, play, and save the synthesized speech as an audio file.
- User-Friendly Interface
    - Provide a simple GUI (e.g., Streamlit) for text input and voice customization.

Suggested Technologies

- Python
- Coqui TTS or Piper TTS
- Streamlit
- Librosa (optional, for audio analysis)
- Pydub or SoundFile (optional, for audio processing)

Performance Evaluation

Students should evaluate their system using the following metrics:

- Mean Opinion Score (MOS): Evaluate the naturalness and quality of the synthesized speech through user feedback.
- Response Time: Measure the time taken to generate speech from the input text.
- Audio Quality: Assess the clarity and intelligibility of the generated speech.
